

Codex Plan: PIJ-OT 消融矩阵 v2 实现方案

2026 因果涌现 / MIGNet Vertical Ablation

2026-06-18

Contents

1	0. 本次开发目标	2
2	1. 现有代码状态判断	2
3	2. 统一数学框架	3
4	3. 基础 cost component 定义	4
4.1	3.1 SR cost	4
4.2	3.2 Pseudotime cost	4
4.3	3.3 Spatial cost	5
4.4	3.4 Expression-derived graph feature cost	5
5	4. 七个方法的 cost 设计	5
5.1	4.1 sr_ot	5
5.2	4.2 pseudotime_ot	6
5.3	4.3 spatial_ot	6
5.4	4.4 sr_spatial_ot	6
5.5	4.5 pseudotime_spatial_ot	7
5.6	4.6 sr_expression_ot	7
5.7	4.7 pseudotime_expression_ot	7
6	5. 推荐的代码结构改动	8
7	6. 具体文件修改清单	8
7.1	6.1 修改 mignet_ce/pij/sr_ot.py	8
7.2	6.2 修改 mignet_ce/pij/pseudotime_ot.py	9
7.3	6.3 新增 mignet_ce/pij/spatial_ot.py	9
7.4	6.4 新增四个组合方法文件	9
7.5	6.5 修改 mignet_ce/pij/registry.py	9
7.6	6.6 修改 mignet_ce/config.py	10
7.6.1	6.6.1 更新 PIJ_METHODS	10
7.6.2	6.6.2 新增主 preset	10
7.6.3	6.6.3 更新旧 preset	10
7.6.4	6.6.4 更新 DEVELOPMENT_PIJ_METHODS	11
8	7. CLI 和运行逻辑要求	11
9	8. 输出和 metadata 要求	11
10	9. 推荐测试计划	12
10.1	9.1 语法测试	12

10.2 9.2 registry 测试	12
10.3 9.3 CLI choices 测试	12
10.4 9.4 最小 smoke run	12
10.5 9.5 结果检查	13
11 10. 验收标准	13
12 11. 给 Codex 的执行顺序	14
13 12. 最重要的建模原则	14

1 0. 本次开发目标

本次改动的目标是重构 PIJ 的 OT 方法消融矩阵。velocity OT 已经被放弃，后续不再作为主实验方法讨论，也不再进入默认消融 preset。新的 OT 消融矩阵围绕两个主发育信号展开：

- SR 信息；
- Pseudotime 信息。

这两个主发育信号需要分别单跑，也就是只使用它们自身的信息来构造 OT cost。然后它们再分别与 spatial 信息、expression-derived graph feature 信息组合。除此之外，需要新增一个 spatial-only OT 作为单跑方法，用来检查空间信息本身对 PIJ 的解释能力。

最终主矩阵包含 7 个 PIJ 方法：

1. **sr_ot**: 只使用 SR / potency 信息；需要 `development_feature_root`，不需要空间坐标，不使用图特征输出。
2. **pseudotime_ot**: 只使用 pseudotime 信息；需要 `development_feature_root`，不需要空间坐标，不使用图特征输出。
3. **spatial_ot**: 只使用 spatial coordinate 信息；不需要 `development_feature_root`，需要空间坐标，不使用图特征输出。
4. **sr_spatial_ot**: 使用 SR / potency + spatial coordinate；需要 `development_feature_root`，需要空间坐标，不使用图特征输出。
5. **pseudotime_spatial_ot**: 使用 pseudotime + spatial coordinate；需要 `development_feature_root`，需要空间坐标，不使用图特征输出。
6. **sr_expression_ot**: 使用 SR / potency + expression-derived graph feature；需要 `development_feature_root`，不需要空间坐标，使用图特征输出。
7. **pseudotime_expression_ot**: 使用 pseudotime + expression-derived graph feature；需要 `development_feature_root`，不需要空间坐标，使用图特征输出。

保留现有输出结构：

```
output/mignet_vertical_ablation_{cluster_method}/
  network={legacy_mixed_grn_cci|cross_cell_multilayer}/
    pij={method_name}/
      metrics.csv
      run_summary.csv
      features/...
```

2 1. 现有代码状态判断

当前 code.zip 里的相关结构如下：

```
mignet_ce/pij/
  _ot_common.py
  expr_ot.py
```

```

energy_entropy_ot.py
pseudotime_ot.py
sr_ot.py
velocity_ot.py
development_ot.py
registry.py

mignet_ce/transition/
  cost_components.py
  ot.py

mignet_ce/io/
  developmental_features.py

mignet_ce/config.py
scripts/run_mignet_vertical.py
scripts/run_mignet_vertical_ablation.py

```

当前问题是：

1. `sr_ot.py` 目前不是纯 SR OT。它同时使用了 expression graph feature cost 和 SR / potency scalar cost。
2. `pseudotime_ot.py` 目前不是纯 pseudotime OT。它同时使用了 expression graph feature cost 和 pseudotime scalar cost。
3. 目前没有 `spatial_ot.py`。
4. 目前没有 `sr_spatial_ot.py`、`pseudotime_spatial_ot.py`、`sr_expression_ot.py`、`pseudotime_expression_ot.py`。
5. `velocity_ot.py` 仍然在 registry 和 preset 中，应从新的主消融 preset 中移除，避免后续误跑。

因此，本次开发不是简单新增几个文件，而是要把旧的 `sr_ot` 和 `pseudotime_ot` 的语义改干净：

```

旧 sr_ot:          expression + SR
新 sr_ot:          SR only
旧 pseudotime_ot:  expression + pseudotime
新 pseudotime_ot:  pseudotime only
旧 sr_ot 的近似语义迁移到:      sr_expression_ot
旧 pseudotime_ot 的近似语义迁移到:  pseudotime_expression_ot

```

3 2. 统一数学框架

设低层级 unit 集合为：

$$\mathcal{U}^{(l)} = \{u_1^{(l)}, u_2^{(l)}, \dots, u_n^{(l)}\}$$

这里 $u_i^{(l)}$ 表示低层第 i 个 unit，例如 spot、less-than-5 domain 或 k150 domain。设高层级 unit 集合为：

$$\mathcal{U}^{(h)} = \{u_1^{(h)}, u_2^{(h)}, \dots, u_m^{(h)}\}$$

这里 $u_j^{(h)}$ 表示高层第 j 个 unit，例如 k150 domain 或 k40 domain。

每一个 OT 方法都构造一个 cost matrix：

$$C \in \mathbb{R}^{n \times m},$$

其中 C_{ij} 表示从低层 unit $u_i^{(l)}$ 映射到高层 unit $u_j^{(h)}$ 的代价。

OT 求解得到 transport plan:

$$\Gamma^* = \arg \min_{\Gamma \in \Pi(a,b)} \sum_{i=1}^n \sum_{j=1}^m \Gamma_{ij} C_{ij} + \varepsilon \sum_{i=1}^n \sum_{j=1}^m \Gamma_{ij} (\log \Gamma_{ij} - 1)$$

这里 Γ_{ij} 表示低层 unit $u_i^{(l)}$ 分配给高层 unit $u_j^{(h)}$ 的 transport mass; ε 是熵正则强度; a 和 b 是低层和高层的边缘分布。当前代码中的 `build_entropic_ot_kernel` 最终会把 transport plan 做 row-normalization, 因此实际导出的 PIJ 是 row-stochastic matrix:

$$P_{ij} = \frac{\Gamma_{ij}^*}{\sum_{j'=1}^m \Gamma_{ij'}^*}$$

于是 P_{ij} 的含义是: 在低层 unit $u_i^{(l)}$ 已知时, 它软映射到高层 unit $u_j^{(h)}$ 的概率。

4 3. 基础 cost component 定义

所有 component 都应该先经过 `normalize_cost`, 也就是归一化到 $[0, 1]$ 。现有代码 `mignet_ce/transition/cost_compor` 已经提供了这些基础函数, 应继续复用, 避免重复实现。

4.1 3.1 SR cost

设低层 unit $u_i^{(l)}$ 的 SR 标量为 $s_i^{(l)}$, 高层 unit $u_j^{(h)}$ 的 SR 标量为 $s_j^{(h)}$ 。SR cost 定义为:

$$D_{ij}^{\text{SR}} = |s_i^{(l)} - s_j^{(h)}|$$

归一化之后:

$$\widetilde{D}_{ij}^{\text{SR}} = \text{Normalize}(D_{ij}^{\text{SR}})$$

实现上继续使用 `pairwise_scalar_cost(source_values, target_values)`。

SR 列的选择规则:

1. 优先使用 `developmental feature` 表中的 `sr` 列;
2. 如果没有 `sr`, 允许 fallback 到 `potency_score`;
3. 如果两个列都没有, 直接报错;
4. 新的纯 `sr_ot` 不应默认混入 `expression cost`, 也不应默认混入 `reverse potency penalty`。

4.2 3.2 Pseudotime cost

设低层 unit $u_i^{(l)}$ 的 pseudotime 为 $\tau_i^{(l)}$, 高层 unit $u_j^{(h)}$ 的 pseudotime 为 $\tau_j^{(h)}$ 。pseudotime cost 定义为:

$$D_{ij}^{\text{PT}} = |\tau_i^{(l)} - \tau_j^{(h)}|$$

归一化之后:

$$\widetilde{D}_{ij}^{\text{PT}} = \text{Normalize}(D_{ij}^{\text{PT}})$$

实现上继续使用 `pairwise_scalar_cost(source_pt, target_pt)`。

新的纯 `pseudotime_ot` 不应默认混入 expression cost, 也不应默认混入 backward pseudotime penalty。若未来要做方向性 pseudotime OT, 应作为新的方法或显式开关, 不要污染这个主消融矩阵。

4.3 3.3 Spatial cost

设低层 unit 的空间坐标为 $\mathbf{r}_i^{(l)}$, 高层 unit 的空间坐标为 $\mathbf{r}_j^{(h)}$ 。空间 cost 定义为:

$$D_{ij}^{\text{SP}} = \|\mathbf{r}_i^{(l)} - \mathbf{r}_j^{(h)}\|_2$$

归一化之后:

$$\widetilde{D}_{ij}^{\text{SP}} = \text{Normalize}(D_{ij}^{\text{SP}})$$

实现上使用已有的 `pairwise_spatial_cost(source_coords, target_coords)`。

如果某个方法包含 spatial component, 但是 `source_coords` 或 `target_coords` 缺失, 应直接报错, 而不是静默退化成其他方法。

4.4 3.4 Expression-derived graph feature cost

这里的 expression 信息不是 raw expression, 而是图表征输出, 也就是 `build_graph_feature_result(...)` 产生的 feature matrix。

设低层 unit 的图特征为 $\mathbf{h}_i^{(l)}$, 高层 unit 的图特征为 $\mathbf{h}_j^{(h)}$ 。默认使用 cosine distance:

$$D_{ij}^{\text{EXP}} = 1 - \frac{\langle \mathbf{h}_i^{(l)}, \mathbf{h}_j^{(h)} \rangle}{\|\mathbf{h}_i^{(l)}\|_2 \|\mathbf{h}_j^{(h)}\|_2 + \delta}$$

归一化之后:

$$\widetilde{D}_{ij}^{\text{EXP}} = \text{Normalize}(D_{ij}^{\text{EXP}})$$

实现上使用已有的 `pairwise_feature_cost(source_features, target_features, metric=cfg.pij_cost_metric)`。

5 4. 七个方法的 cost 设计

5.1 4.1 sr_ot

只使用 SR 信息:

$$C_{ij}^{\text{SR}} = \widetilde{D}_{ij}^{\text{SR}}$$

代码要求:

- 修改现有 `mignet_ce/pij/sr_ot.py`;
- 删除默认 expression component;
- 不再把 `source_features` 和 `target_features` 纳入 cost;
- 只读取 developmental feature 表里的 `sr` 或 `potency_score`;
- `weights` 中只包含当前使用的 scalar component。

5.2 4.2 pseudotime_ot

只使用 pseudotime 信息:

$$C_{ij}^{\text{PT}} = \widetilde{D}_{ij}^{\text{PT}}$$

代码要求:

- 修改现有 `mignet_ce/pij/pseudotime_ot.py`;
- 删除默认 expression component;
- 不再把 `source_features` 和 `target_features` 纳入 cost;
- 只读取 developmental feature 表中的 `pseudotime`;
- `weights` 中只包含 `pseudotime`。

5.3 4.3 spatial_ot

只使用空间坐标信息:

$$C_{ij}^{\text{SP}} = \widetilde{D}_{ij}^{\text{SP}}$$

代码要求:

- 新增 `mignet_ce/pij/spatial_ot.py`;
- 不读取 developmental feature root;
- 不使用 expression graph feature;
- 只使用 `_ot_common.py` 传入的 `source_coords` 和 `target_coords`;
- 如果坐标缺失, 报错: `spatial_ot requires coordinates.`

5.4 4.4 sr_spatial_ot

同时使用 SR 和 spatial:

$$C_{ij}^{\text{SR+SP}} = \frac{w_{\text{SR}} \widetilde{D}_{ij}^{\text{SR}} + w_{\text{SP}} \widetilde{D}_{ij}^{\text{SP}}}{w_{\text{SR}} + w_{\text{SP}}}$$

第一版主实验建议设置:

$$w_{\text{SR}} = 1, \quad w_{\text{SP}} = 1$$

代码要求:

- 新增 `mignet_ce/pij/sr_spatial_ot.py`;
- 读取 `sr` 或 `potency_score`;
- 同时读取空间坐标;
- 不使用 expression graph feature;
- `metadata` 中记录使用了 `sr` 还是 `potency_score`。

5.5 4.5 pseudotime_spatial_ot

同时使用 pseudotime 和 spatial:

$$C_{ij}^{\text{PT+SP}} = \frac{w_{\text{PT}} \widetilde{D}_{ij}^{\text{PT}} + w_{\text{SP}} \widetilde{D}_{ij}^{\text{SP}}}{w_{\text{PT}} + w_{\text{SP}}}$$

第一版主实验建议设置:

$$w_{\text{PT}} = 1, \quad w_{\text{SP}} = 1$$

代码要求:

- 新增 `mignet_ce/pij/pseudotime_spatial_ot.py`;
- 读取 `pseudotime`;
- 同时读取空间坐标;
- 不使用 `expression graph feature`。

5.6 4.6 sr_expression_ot

同时使用 SR 和 expression-derived graph feature:

$$C_{ij}^{\text{SR+EXP}} = \frac{w_{\text{SR}} \widetilde{D}_{ij}^{\text{SR}} + w_{\text{EXP}} \widetilde{D}_{ij}^{\text{EXP}}}{w_{\text{SR}} + w_{\text{EXP}}}$$

第一版主实验建议设置:

$$w_{\text{SR}} = 1, \quad w_{\text{EXP}} = 1$$

代码要求:

- 新增 `mignet_ce/pij/sr_expression_ot.py`;
- 读取 `sr` 或 `potency_score`;
- 使用 `source_features` 和 `target_features` 构造 `expression-derived graph feature cost`;
- 不使用 `spatial coordinate`。

5.7 4.7 pseudotime_expression_ot

同时使用 pseudotime 和 expression-derived graph feature:

$$C_{ij}^{\text{PT+EXP}} = \frac{w_{\text{PT}} \widetilde{D}_{ij}^{\text{PT}} + w_{\text{EXP}} \widetilde{D}_{ij}^{\text{EXP}}}{w_{\text{PT}} + w_{\text{EXP}}}$$

第一版主实验建议设置:

$$w_{\text{PT}} = 1, \quad w_{\text{EXP}} = 1$$

代码要求:

- 新增 `mignet_ce/pij/pseudotime_expression_ot.py`;
- 读取 `pseudotime`;

- 使用 `source_features` 和 `target_features` 构造 expression-derived graph feature cost;
- 不使用 spatial coordinate。

6 5. 推荐的代码结构改动

为了避免 6 个 developmental OT 文件重复写 feature loading 逻辑，建议新增一个内部 helper 文件：

`mignet_ce/pij/_developmental_ot_components.py`

这个文件只服务 PIJ 方法内部，不作为公开 API。建议提供以下 helper：

```
def make_developmental_table_loader(context, cfg):
    ...

def build_sr_cost(source_table, target_table, method_name):
    ...

def build_pseudotime_cost(source_table, target_table, method_name):
    ...

def build_spatial_cost(source_coords, target_coords, method_name):
    ...

def build_expression_cost(source_features, target_features, cfg):
    ...

def developmental_metadata(source_table, target_table, used_columns):
    ...
```

其中：

- `make_developmental_table_loader` 负责复用现有 `load_developmental_features_for_pij(context, cfg, time_index, space)`，并做 cache；
- `build_sr_cost` 内部优先选择 sr，fallback 到 `potency_score`；
- `build_pseudotime_cost` 必须要求 `pseudotime`；
- `build_spatial_cost` 必须在坐标缺失时报错；
- `build_expression_cost` 直接调用 `pairwise_feature_cost`。

这样每一个方法文件只需要保留自己的 cost 组合逻辑，不要把读取、校验、metadata 拼装写得到处都是。

7 6. 具体文件修改清单

7.1 6.1 修改 `mignet_ce/pij/sr_ot.py`

目标：让 `sr_ot` 从 “SR + expression” 改为 “SR only”。

核心 component builder 应该变成类似语义：

```
components = {
    component_name: pairwise_scalar_cost(source_values, target_values),
}
weights = {
    component_name: 1.0,
}
```

不要再出现：

```
"expression": pairwise_feature_cost(...)
```

7.2 6.2 修改 mignet_ce/pij/pseudotime_ot.py

目标：让 pseudotime_ot 从 “pseudotime + expression” 改为 “pseudotime only”。

核心 component builder 应该变成类似语义：

```
components = {  
    "pseudotime": pairwise_scalar_cost(source_pt, target_pt),  
}  
weights = {  
    "pseudotime": 1.0,  
}
```

不要再出现：

```
"expression": pairwise_feature_cost(...)
```

7.3 6.3 新增 mignet_ce/pij/spatial_ot.py

新增类：

```
class SpatialOTPijMethod:  
    name = "spatial_ot"
```

component builder 只返回：

```
components = {  
    "spatial": pairwise_spatial_cost(source_coords, target_coords),  
}  
weights = {  
    "spatial": 1.0,  
}
```

7.4 6.4 新增四个组合方法文件

新增：

```
mignet_ce/pij/sr_spatial_ot.py  
mignet_ce/pij/pseudotime_spatial_ot.py  
mignet_ce/pij/sr_expression_ot.py  
mignet_ce/pij/pseudotime_expression_ot.py
```

每个文件只做自己的 cost 组合，不要复写 OT 求解。OT 求解仍然统一交给：

```
run_ot_pij_method(...)
```

7.5 6.5 修改 mignet_ce/pij/registry.py

增加 import：

```
from mignet_ce.pij.spatial_ot import SpatialOTPijMethod  
from mignet_ce.pij.sr_spatial_ot import SRSpatialOTPijMethod  
from mignet_ce.pij.pseudotime_spatial_ot import PseudotimeSpatialOTPijMethod  
from mignet_ce.pij.sr_expression_ot import SRExpressionOTPijMethod  
from mignet_ce.pij.pseudotime_expression_ot import PseudotimeExpressionOTPijMethod
```

注册：

```

"spatial_ot": SpatialOTPiMethod,
"sr_spatial_ot": SRSpatialOTPiMethod,
"pseudotime_spatial_ot": PseudotimeSpatialOTPiMethod,
"sr_expression_ot": SRExpressionOTPiMethod,
"pseudotime_expression_ot": PseudotimeExpressionOTPiMethod,

```

velocity_ot 可以暂时保留在 registry 里用于兼容旧命令, 但不能进入新的主 preset。如果想更严格, 也可以从 registry 中移除, 但这样旧命令会直接报 unsupported method。第一版建议: 保留文件和 registry, 移出 preset。

7.6 6.6 修改 mignet_ce/config.py

7.6.1 6.6.1 更新 PIJ_METHODS

加入:

```

"spatial_ot",
"sr_spatial_ot",
"pseudotime_spatial_ot",
"sr_expression_ot",
"pseudotime_expression_ot",

```

7.6.2 6.6.2 新增主 preset

新增:

```

"ot_ablation_v2": (
    "sr_ot",
    "pseudotime_ot",
    "spatial_ot",
    "sr_spatial_ot",
    "pseudotime_spatial_ot",
    "sr_expression_ot",
    "pseudotime_expression_ot",
),

```

7.6.3 6.6.3 更新旧 preset

建议修改:

```

"development_scalar": ("pseudotime_ot", "sr_ot"),

```

建议新增或修改:

```

"development_ot_ablation": (
    "sr_ot",
    "pseudotime_ot",
    "spatial_ot",
    "sr_spatial_ot",
    "pseudotime_spatial_ot",
    "sr_expression_ot",
    "pseudotime_expression_ot",
),

```

development_all 和 all 中不要再默认包含 velocity_ot。如果为了兼容不想动 all, 至少要确保新的实验命令不使用 all, 而是使用 ot_ablation_v2。

7.6.4 6.6.4 更新 DEVELOPMENT_PIJ_METHODS

需要 developmental feature root 的方法包括:

```
DEVELOPMENT_PIJ_METHODS = {
    "pseudotime_ot",
    "sr_ot",
    "sr_spatial_ot",
    "pseudotime_spatial_ot",
    "sr_expression_ot",
    "pseudotime_expression_ot",
    "development_ot",
}
```

注意:

- spatial_ot 不需要 developmental feature root;
- expr_ot 不需要 developmental feature root;
- velocity_ot 不进入新集合, 除非保留旧兼容逻辑。

8 7. CLI 和运行逻辑要求

run_mignet_vertical.py 和 run_mignet_vertical_ablation.py 当前已经从 PIJ_METHODS 和 PIJ_METHOD_PRESETS 读取 choices, 因此只要 config.py 更新正确, CLI choices 会自动更新。

需要确保以下命令可以解析:

```
python scripts/run_mignet_vertical_ablation.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_ablation_louvain" \
--network-methods legacy_mixed_grn_cci cross_cell_multilayer \
--pij-method-preset ot_ablation_v2 \
--pair-preset louvain_all \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" \
--export-pij
```

如果只跑 spatial-only control, 可以允许不传 --development-feature-root:

```
python scripts/run_mignet_vertical.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_ablation_louvain/network" \
--network-method legacy_mixed_grn_cci \
--pij-method spatial_ot \
--pair-preset louvain_all \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--export-pij
```

9 8. 输出和 metadata 要求

每个 PIJ 方法都必须在 kernel_metadata.json 中能看出以下信息:

1. pij_method;
2. cost_components, 例如 ['sr']、['pseudotime', 'spatial'];

3. 每个 component 的 weight;
4. 每个 component 的 min、max、mean;
5. 如果用了 developmental feature, 需要记录 source / target feature CSV 的 metadata;
6. 如果 SR 使用的是 fallback potency_score, 必须在 metadata 中写清楚。

transition_topk_table 已经会把 diagnostics cost 写进 top-k 表。新增方法应该继续让 kernel_diagnostics 包含:

```
sr_cost
pseudotime_cost
spatial_cost
expression_cost
main_cost
```

但不同方法只导出自己真的使用的 cost。例如 sr_ot 不应该出现 expression_cost。

10 9. 推荐测试计划

10.1 9.1 语法测试

在项目根目录运行:

```
python -m compileall mignet_ce scripts
```

10.2 9.2 registry 测试

运行一个小脚本检查新方法都能被 registry 识别:

```
python - <<'PY'
from mignet_ce.pij.registry import get_pij_method
methods = [
    "sr_ot",
    "pseudotime_ot",
    "spatial_ot",
    "sr_spatial_ot",
    "pseudotime_spatial_ot",
    "sr_expression_ot",
    "pseudotime_expression_ot",
]
for name in methods:
    method = get_pij_method(name)
    assert method.name == name
print("all new OT methods are registered")
PY
```

10.3 9.3 CLI choices 测试

检查 CLI 是否接受新 preset:

```
python scripts/run_mignet_vertical_ablation.py --help | grep ot_ablation_v2
python scripts/run_mignet_vertical.py --help | grep spatial_ot
```

10.4 9.4 最小 smoke run

先只跑一个 organ、两个时间点、一个 pair:

```
python scripts/run_mignet_vertical.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/EIS1_domain_factory" \
--output-root "/home/jovyan/work/2026 Causality/output/smoke_pij_ot_v2/pij=sr_ot" \
--network-method legacy_mixed_grn_cci \
--pij-method sr_ot \
--organs heart \
--time-points 11.5 12.5 \
--level-pairs spot:louvain_less_than5 \
--development-feature-root "/home/jovyan/work/2026 Causality/output/developmental_features" \
--export-pij
```

然后分别替换 --pij-method 为:

```
pseudotime_ot
spatial_ot
sr_spatial_ot
pseudotime_spatial_ot
sr_expression_ot
pseudotime_expression_ot
```

其中 spatial_ot 不需要 --development-feature-root, 其他六个需要。

10.5 9.5 结果检查

对每个 smoke run 检查:

```
metrics.csv 存在
run_summary.csv 存在
features/{organ}/{pair}/pij/kernel_metadata.json 存在
features/{organ}/{pair}/pij/*_P_topk.csv 存在
```

并检查每个 PIJ 矩阵是否 row-stochastic:

```
import numpy as np
from scipy import sparse

matrix = sparse.load_npz("../lower_P.npz").toarray()
assert np.allclose(matrix.sum(axis=1), 1.0, atol=1e-5)
```

11 10. 验收标准

本次开发完成后, 应满足以下标准:

1. sr_ot 的 metadata 里只出现 SR / potency cost, 不出现 expression cost。
2. pseudotime_ot 的 metadata 里只出现 pseudotime cost, 不出现 expression cost。
3. spatial_ot 可以独立运行, 不依赖 developmental feature root。
4. sr_spatial_ot 只包含 SR / potency cost 与 spatial cost。
5. pseudotime_spatial_ot 只包含 pseudotime cost 与 spatial cost。
6. sr_expression_ot 只包含 SR / potency cost 与 expression graph feature cost。
7. pseudotime_expression_ot 只包含 pseudotime cost 与 expression graph feature cost。
8. ot_ablation_v2 preset 包含上述七个方法。
9. velocity_ot 不再出现在新的主实验 preset 中。
10. 所有新方法输出路径继续遵守 network=.../pij=... 的结构。

12 11. 给 Codex 的执行顺序

建议 Codex 按如下顺序实现：

1. 新建 `_developmental_ot_components.py`，先抽出 SR、pseudotime、spatial、expression 四类 cost builder。
2. 修改 `sr_ot.py`，使其变成 SR-only。
3. 修改 `pseudotime_ot.py`，使其变成 pseudotime-only。
4. 新增 `spatial_ot.py`。
5. 新增四个组合方法文件。
6. 修改 `registry.py`。
7. 修改 `config.py` 的 `PIJ_METHODS`、`PIJ_METHOD_PRESETS`、`DEVELOPMENT_PIJ_METHODS`。
8. 运行 `python -m compileall mignet_ce scripts`。
9. 运行 registry 测试。
10. 运行最小 smoke run。
11. 检查 `kernel_metadata.json`，确认 cost components 与方法语义完全一致。

13 12. 最重要的建模原则

本次消融的核心原则是：

OT 求解器、熵正则、PIJ 归一化方式、输出结构全部保持一致，只改变 cost matrix 的信息来源。

因此，不同方法之间的实验差异才能解释为不同信息源对跨层级 PIJ 的贡献差异。

特别注意：

- `sr_ot` 不再是 SR + expression；
- `pseudotime_ot` 不再是 pseudotime + expression；
- expression graph feature 的组合实验必须显式写成 `sr_expression_ot` 或 `pseudotime_expression_ot`；
- `spatial` 单跑必须显式写成 `spatial_ot`；
- velocity OT 不进入本轮消融矩阵。